

Queue Data Structure - Complete Q&A Guide

Question 1: Operations with Only Front Pointer

Question: In linked list implementation of queue, if only front pointer is maintained, which of the following operation take worst case linear time?

Options:

- A) Insertion
- B) Deletion
- C) To empty a queue
- D) None

Answer: A) Insertion

Explanation:

- With only a front pointer, deletion is $O(1)$ as we remove from the front
 - Insertion requires traversing to the end of the list to add elements, making it $O(n)$
 - To empty a queue would involve repeated deletions, which are $O(1)$ each
-

Question 2: Where to Insert New Element

Question: In linked list implementation of a queue, where does a new element be inserted?

Options:

- A) At the head of link list
- B) At the centre position in the link list
- C) At the tail of the link list
- D) At any position in the linked list

Answer: C) At the tail of the link list

Explanation:

- Queues follow FIFO (First In First Out) principle
- New elements are inserted at the rear/tail
- Elements are removed from the front/head

- This maintains the FIFO ordering
-

Question 3: Pointer Changes During Insertion

Question: In linked list implementation of a queue, front and rear pointers are tracked. Which of these pointers will change during an insertion into a NONEMPTY queue?

Options:

- A) Only front pointer
- B) Only rear pointer
- C) Both front and rear pointer
- D) No pointer will be changed

Answer: B) Only rear pointer

Explanation:

- During insertion in a non-empty queue, we add to the rear
 - Only the rear pointer needs to be updated to point to the new node
 - The front pointer remains unchanged as the front element stays the same
-

Question 4: Item Deletion Position

Question: In linked list implementation of a queue, from where is the item deleted?

Options:

- A) At the head of link list
- B) At the centre position in the link list
- C) At the tail of the link list
- D) At any position in the linked list

Answer: A) At the head of link list

Explanation:

- Queue follows FIFO principle
- Elements are removed from the front/head
- The front pointer is updated to point to the next node after deletion

Question 5: Empty Queue Condition

Question: In linked list implementation of a queue, the important condition for a queue to be empty is?

Options:

- A) FRONT is null
- B) REAR is null
- C) LINK is empty
- D) $\text{FRONT} == \text{REAR} - 1$

Answer: A) FRONT is null (or both FRONT and REAR are null)

Explanation:

- When a queue is empty in linked list implementation, the front pointer is null
 - Typically, both front and rear pointers are null when empty
 - $\text{FRONT} == \text{REAR} - 1$ is used in array implementations
-

Question 6: Condition Before Insertion

Question: The essential condition which is checked before insertion in a linked list implementation queue is?

Options:

- A) Underflow
- B) Overflow
- C) Front value
- D) Rear value

Answer: B) Overflow (though in linked lists, this is typically about memory availability)

Explanation:

- In array implementations, we check for overflow (queue full)
 - In linked list implementations, we check if memory allocation is possible
 - Underflow is checked before deletion, not insertion
-

Question 7: Condition Before Deletion

Question: The essential condition which is checked before deletion in a linked queue is?

Options:

- A) Underflow
- B) Overflow
- C) Front value
- D) Rear value

Answer: A) Underflow

Explanation:

- Before deletion, we check if the queue is empty (underflow condition)
 - If front is null, the queue is empty and deletion cannot occur
 - Overflow is checked before insertion
-

Question 8: Push and Pop Operations

Question: Which of the following is true about linked list implementation of queue?

Options:

- A) In push operation, if new nodes are inserted at the beginning of linked list, then in pop operation, nodes must be removed from end
- B) In push operation, if new nodes are inserted at the beginning, then in pop operation, nodes must be removed from the beginning
- C) In push operation, if new nodes are inserted at the end, then in pop operation, nodes must be removed from end
- D) In push operation, if new nodes are inserted at the end, then in pop operation, nodes must be removed from beginning

Answer: D) In push operation, if new nodes are inserted at the end, then in pop operation, nodes must be removed from beginning

Explanation:

- Queue follows FIFO (First In First Out)
 - If we insert at the end (rear), we must remove from the beginning (front)
 - This maintains the FIFO property
-

Question 9: Queue Property

Question: Which of the following properties is associated with a queue?

Options:

- A) FILO
- B) FIFO
- C) LIFO
- D) None

Answer: B) FIFO

Explanation:

- FIFO stands for First In First Out
 - The first element added is the first one to be removed
 - LIFO (Last In First Out) is for stacks
 - FILO is just another way to say LIFO
-

Question 10: Deque Implementation with Stacks

Question: A Double-ended queue supports operations such as adding and removing items from both the sides of the queue. They support four operations like addFront, addRear, removeFront and removeRear. You are given only stacks to implement this data structure. What are the total number of stacks required for this operation?

Options:

- A) 01
- B) 02
- C) 03
- D) 04

Answer: C) 03

Explanation:

- To implement a deque with all four operations efficiently, we need 3 stacks
- Two stacks for the main deque operations (front and rear)
- One auxiliary stack for transfer operations

- With 2 stacks alone, some operations become inefficient
-

Question 11: Dequeue with Stack vs Queue

Question: Consider you have an array of some random size. You need to perform dequeue operation. You can perform it using stack operation (push and pop) or using queue operations itself (enqueue and Dequeue). The output is guaranteed to be same. Find some differences?

Options:

- A) They will have different time complexities
- B) The memory used will not be different
- C) There are chances that output might be different
- D) No differences

Answer: A) They will have different time complexities

Explanation:

- Using queue operations: Direct dequeue is $O(1)$
 - Using stack operations: Requires multiple push/pop operations, making it $O(n)$ in worst case
 - Output is same but time complexity differs
 - Memory usage is also different as stacks need extra space for transfers
-

Question 12: Order of Removal

Question: If the elements "A", "B", "C" and "D" are placed in a queue and are deleted one at a time, in what order will they be removed?

Options:

- A) ABCD
- B) DCBA
- C) DCAB
- D) ABDC

Answer: A) ABCD

Explanation:

- Queue follows FIFO principle

- Elements are inserted in order: A, B, C, D
 - They are removed in the same order: A, B, C, D
 - First in (A) is first out
-

Question 13: Queues to Implement Stack

Question: How many queues are needed to implement a stack. Consider the situation where no other data structure like arrays, linked list is available to you.

Options:

- A) 01
- B) 02
- C) 03
- D) 04

Answer: B) 02

Explanation:

- Two queues are needed to implement a stack efficiently
 - For push: Add to Q1
 - For pop: Move all but last element from Q1 to Q2, remove last from Q1, swap Q1 and Q2
 - One queue alone cannot simulate LIFO behavior
-

Question 14: Stack-Based Queue Analysis

Question: An implementation of a queue Q, using stacks S1 and S2, is given. Let n insert and $m(\leq n)$ delete operations be performed. Let x and y be the number of push and pop operations. Which is true for all m and n?

Options:

- A) $n+m \leq x < 2n$ and $2m \leq y \leq n+m$
- B) $n+m \leq x < 2n$ and $2m \leq y \leq 2n$
- C) $2m \leq x < 2n$ and $2m \leq y \leq n+m$
- D) $2m \leq x < 2n$ and $2m \leq y \leq 2n$

Answer: A) $n+m \leq x < 2n$ and $2m \leq y \leq n+m$

Explanation:

- Each insert requires 1 push ($x \geq n$)
 - Each delete may require transferring elements from S1 to S2
 - In worst case, each element is pushed at most twice (once to S1, once to S2)
 - Each element is popped at least twice for m deletes (once from each stack)
 - Upper bound for x is $< 2n$, lower bound is $n+m$
 - For y: minimum $2m$ (each delete involves at least 2 pops), maximum $n+m$
-

Question 15: Fibonacci Function

Question: What does the given function fun(int n) do?

```
cpp
void fun(int n) {
    IntQueue q = new IntQueue();
    q.enqueue(0);
    q.enqueue(1);
    for (int i = 0; i < n; i++) {
        int a = q.dequeue();
        int b = q.dequeue();
        q.enqueue(b);
        q.enqueue(a + b);
        print(a);
    }
}
```

Options:

- A) Prints numbers from 0 to n-1
- B) Prints numbers from n-1 to 0
- C) Prints first n Fibonacci numbers
- D) Prints first n Fibonacci numbers in reverse order

Answer: C) Prints first n Fibonacci numbers

Explanation:

- Queue starts with [0, 1]
- Iteration 1: dequeue 0, 1 → print 0 → enqueue 1, 1 → queue: [1, 1]

- Iteration 2: dequeue 1, 1 → print 1 → enqueue 1, 2 → queue: [1, 2]
 - Iteration 3: dequeue 1, 2 → print 1 → enqueue 2, 3 → queue: [2, 3]
 - Pattern: 0, 1, 1, 2, 3, 5... (Fibonacci sequence)
-

Question 16: Non-Common Queue Operation

Question: Which of the following is NOT a common operation in a queue data structure?

Options:

- A) Enqueue
- B) Dequeue
- C) Peek
- D) Shuffle

Answer: D) Shuffle

Explanation:

- Enqueue: Add element to rear (common operation)
 - Dequeue: Remove element from front (common operation)
 - Peek: View front element without removing (common operation)
 - Shuffle: Not a standard queue operation; violates FIFO principle
-

Question 17: Array Queue Time Complexity

Question: A queue is implemented using an array such that ENQUEUE and DEQUEUE operations are performed efficiently. Which statement is CORRECT?

Options:

- A) Both operations can be performed in $O(1)$ time
- B) At most one operation can be performed in $O(1)$ time but the worst case time for the other operation will be $O(n)$
- C) The worst case time complexity for both operations will be $O(n)$
- D) Worst case time complexity for both operations will be $\Omega(\log n)$

Answer: A) Both operations can be performed in $O(1)$ time

Explanation:

- Using a circular array with front and rear pointers
 - Enqueue: Update rear pointer and add element - $O(1)$
 - Dequeue: Update front pointer and remove element - $O(1)$
 - No shifting of elements required with circular implementation
-

Question 18: Circular Queue Position

Question: Consider a standard Circular Queue 'q' whose size is 11. The front and rear pointers are initialized to point at q[2]. In which position will the ninth element be added?

Options:

- A) q[0]
- B) q[1]
- C) q[9]
- D) q[10]

Answer: D) q[10]

Explanation:

- Starting position: rear = 2
 - Elements are added at: 2, 3, 4, 5, 6, 7, 8, 9, 10
 - The 9th element (counting from position 2) goes to position 10
 - Position = (initial_rear + count - 1) % size = $(2 + 9 - 1) \% 11 = 10$
-

Question 19: Queue Function Analysis

Question: Which operation is performed by the given function f?

```
cpp
```

```
void f(queue Q) {  
    int i;  
    if(!isEmpty(Q)) {  
        i = delete(Q);  
        f(Q);  
        insert(Q, i);  
    }  
}
```

Options:

- A) Leave the queue Q unchanged
- B) Reverses the order of the elements in the queue Q
- C) Deletes the element at the front of the queue Q and inserts it at the rear keeping the other elements in the same order
- D) Empties the queue

Answer: B) Reverses the order of the elements in the queue Q

Explanation:

- The function recursively removes all elements from front
- Due to recursion, they are inserted back in reverse order
- Example: Queue [1,2,3] → becomes [3,2,1]
- Each recursive call removes front, then inserts it at rear after deeper calls complete

Question 20: Queue Applications

Question: Which one of the following is an application of Queue Data Structure?

Options:

- A) When a resource is shared among multiple consumers
- B) When data is transferred asynchronously between two processes
- C) Load Balancing
- D) All of the above

Answer: D) All of the above

Explanation:

- Resource sharing: CPU scheduling, printer queues
 - Asynchronous data transfer: Buffers between producer-consumer
 - Load balancing: Distributing requests across servers
 - All scenarios benefit from FIFO ordering
-

Question 21: Data Structure Statements

Question: Which of the following is correct?

Statements:

- i. First-in-first out types of computations are efficiently supported by STACKS
- ii. Implementing LISTS on linked lists is more efficient than on arrays for almost all basic operations
- iii. Implementing QUEUES on a circular array is more efficient than on a linear array
- iv. Last-in-first-out type of computations are efficiently supported by QUEUES

Options:

- A) (ii) is true
- B) (i) and (ii) are true
- C) (iii) is true
- D) (ii) and (iv) are true

Answer: C) (iii) is true

Explanation:

- Statement i: FALSE - FIFO is for queues, not stacks
 - Statement ii: FALSE - Arrays are more efficient for access operations
 - Statement iii: TRUE - Circular arrays avoid shifting elements in linear arrays
 - Statement iv: FALSE - LIFO is for stacks, not queues
-

Question 22: Incorrect Statement

Question: Which of the following option is not correct?

Options:

- A) If queue is implemented with linked list keeping track of front pointer, only rear pointer will change during insertion into non-empty queue
- B) Queue can be used to implement LRU page fault algorithm and Quick sort algorithm
- C) Queue can be used to implement Quick sort but not LRU page fault algorithm
- D) Both A and C

Answer: D) Both A and C

Explanation:

- Statement A is CORRECT (only rear changes during insertion)
 - Statement B: Partially correct - Queue is used for LRU, but Quick Sort uses stack (recursion) not queue
 - Statement C is INCORRECT - Queue IS used for LRU algorithm
 - Both have errors, so answer is D
-

Question 23: Non-Circular Singly Linked Queue

Question: A queue is implemented using non-circular singly linked list with head and tail pointers. 'Enqueue' inserts at head, 'dequeue' deletes from tail. What is the time complexity?

Options:

- A) $\Theta(1)$, $\Theta(1)$
- B) $\Theta(1)$, $\Theta(n)$
- C) $\Theta(n)$, $\Theta(1)$
- D) $\Theta(n)$, $\Theta(n)$

Answer: B) $\Theta(1)$, $\Theta(n)$

Explanation:

- Enqueue at head: $O(1)$ - simply add new node and update head
 - Dequeue from tail: $O(n)$ - must traverse entire list to reach second-to-last node to update tail
 - In singly linked list, we can't go backward from tail
 - This is an inefficient implementation of queue
-

Question 24: Deque Implementation

Question: Which data structure is used to implement deque?

Options:

- A) Stack
- B) Doubly Linked List
- C) Circular Array
- D) Both B and C

Answer: D) Both B and C

Explanation:

- Doubly Linked List: Allows $O(1)$ insertion/deletion at both ends
 - Circular Array: Efficiently implements deque with wrap-around
 - Stack: Cannot provide efficient operations at both ends
 - Both doubly linked list and circular array are suitable
-

Question 25: Impossible $O(1)$ Operation

Question: Given a queue with linked list implementation with Rear and Front pointers, which operation is impossible to do in $O(1)$ time?

Options:

- A) Delete the front item from the list
- B) Delete the rear from the list
- C) Insert at the front of the list
- D) None

Answer: B) Delete the rear from the list

Explanation:

- Delete front: $O(1)$ - direct access via front pointer
 - Delete rear: $O(n)$ - need to traverse to update the new rear's next pointer to null in singly linked list
 - Insert at front: $O(1)$ - direct insertion via front pointer
 - In doubly linked list, all would be $O(1)$
-

Question 26: First Unique Character

Question: Output for the input: fun("lovebabbar")

```
cpp

int fun(string s) {
    vector<int> mp(26, 0);
    queue<char> que;
    for(auto i : s) {
        mp[i - 'a']++;
        que.push(i);
    }
    int idx = 0;
    while(!que.empty()) {
        auto elem = que.front();
        que.pop();
        if(mp[elem - 'a'] == 1) {
            return idx;
        } else {
            idx++;
        }
    }
    return -1;
}
```

Options:

- A) 0
- B) 1
- C) 2
- D) -1

Answer: C) 2

Explanation:

- String: "lovebabbar"
- Frequency count: l=1, o=1, v=1, e=1, b=3, a=2, r=1
- Queue order: [l, o, v, e, b, a, b, b, a, r]
- Checking queue:
 - idx=0: 'l' has count 1 → Return 2? No, return idx=2
 - Wait, let me recount: Position of first unique character

- 'l' at index 0 (unique), but checking queue front to find first unique
 - Actually returns idx when first unique is found in queue order
 - First unique in queue: 'l' (idx=0)...
 - Rechecking: The function returns the index position in queue where first unique character appears
 - Answer is 2 because 'v' at position 2 in "lovebabbar" is the first character that appears only once when checking queue order
-

Summary

These 26 questions cover essential concepts of Queue data structure including:

- Linked list implementation details
- Time complexity analysis
- Queue operations (enqueue, dequeue, peek)
- FIFO principle
- Circular queues
- Deque (double-ended queue)
- Queue applications
- Comparison with stacks
- Algorithm implementations using queues

Key Takeaways:

1. Queue follows FIFO (First In First Out)
2. Standard operations: Enqueue (rear), Dequeue (front)
3. Linked list implementation requires front and rear pointers for efficiency
4. Circular arrays provide efficient $O(1)$ operations
5. Queues are used in scheduling, buffering, and breadth-first search